

Chapter Four

Array

Arrays

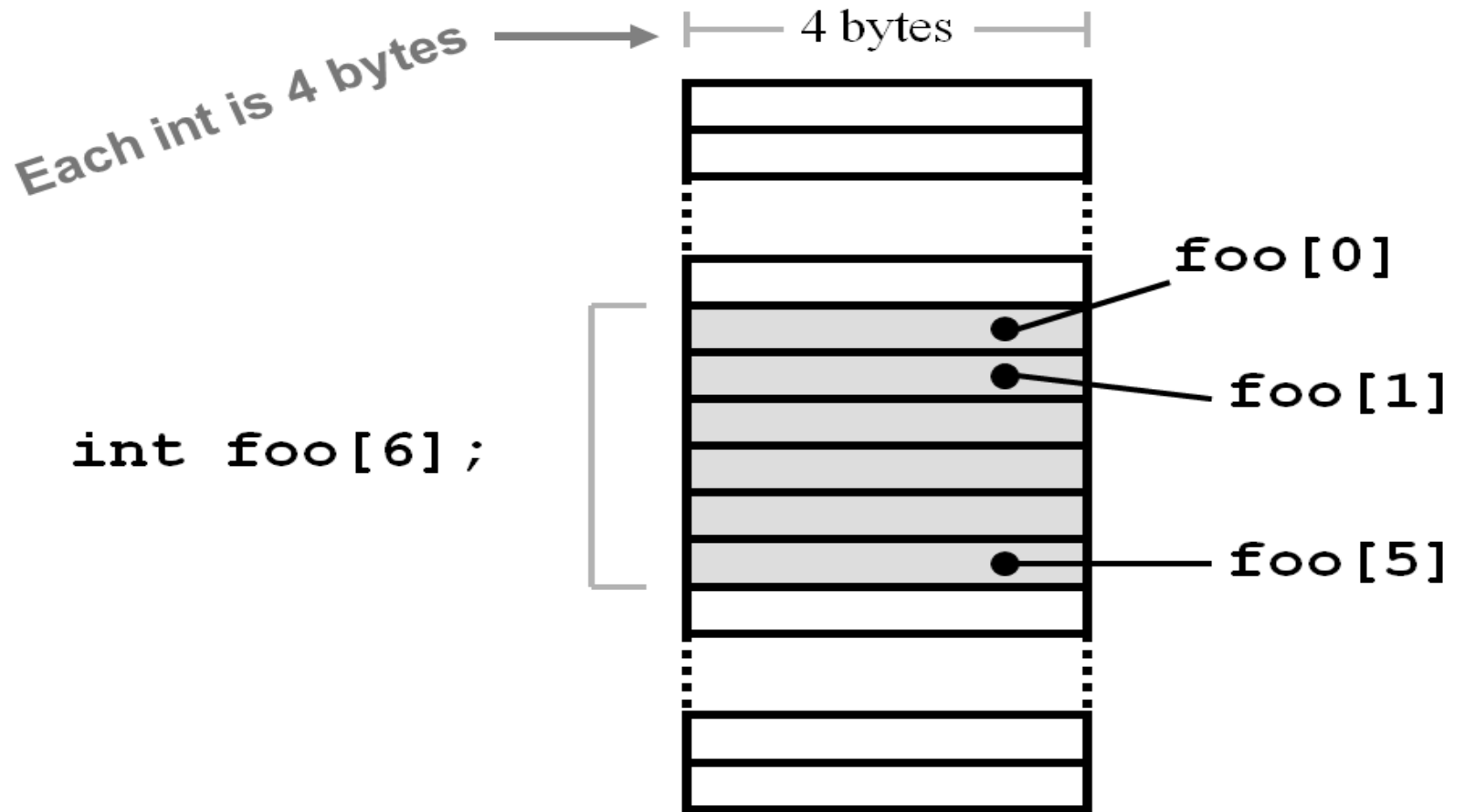
- Introduction
- Implementing arrays
- Multidimensional arrays
- Strings and arrays of characters
- Passing arrays to functions

Introduction

- An array is a consecutive group of memory locations.
- Each *group* is called an element of the array.
- The contents of each element are of the same *type*.
 - Could be an array of int, double, char, ...
- We can refer to individual *elements* by giving the position number (index) of the element in the array.

Introduction

- Memory and Arrays



Introduction

- **Array Subscripts**

- The element numbers are called subscripts.


The diagram shows the expression `foo[i]`. A light yellow arrow points from the text "Array name" to the `foo` part. Another light yellow arrow points from the text "subscript" to the `i` part.

- A subscript can be any integer expression:
- These are all valid subscripts:

– `foo[17]` `foo[i+3]` `foo[a+b+c]`

Implementing arrays

- Array implementation may consists of operations such as array declaration, array initialization, array processing(Input data, Output data, stored in an array, Find the largest and/or smallest element)
 - Each operation requires ability to step through the elements of the array
 - Easily accomplished by a loop
- Array declaration
 - `element_type array_name[number_of_elements];`
 - `element_type` can be any C++ variable type.
 - `array_name` can be any valid variable name.
 - `number_of_elements` can be an expression.

- Array initialization
 - You can initialize an array when you declare it (just like with variables):
 - `int foo[5] = { 1, 8, 3, 6, 12};`
 - `double d[2] = { 0.707, 0.707};`
 - `char s[] = { 'R', 'P', 'I' };`
 - You don't need to specify a size when initializing, the compiler will count for you.
- Partial Initialization
 - The statement
`int list[10] = {0};`
declares `list` to be an array of 10 components and initializes all components to zero
 - The statement
`int list[10] = {8, 5, 12};`
declares `list` to be an array of 10 components, initializes `list[0]` to 8, `list[1]` to 5, `list[2]` to 12 and all other components are initialized to 0

Implementing arrays

- Accessing Array Components

- Consider the declaration

```
int list[100]; //list is an array
int i; //of the size 100
```

- This for loop steps through each element of the array list starting at the first element

```
for (i = 0; i < 100; i++)
    //the process is input data
    cin >> list[i];
```

- Accessing Array Components

```
double sales[10];  
int index;  
double largestSale, sum, average;
```

- Initialize an array

```
for (index = 0; index < 10; index++)  
    sales[index] = 0.0;
```

- Read data into an array

```
for (index = 0; index < 10; index++)  
    cin >> sales[index];
```

- Printing an array

```
for (index = 0; index < 10; index++)  
    cout << sales[index] << " ";
```

Implementing arrays

- Accessing Array Components
- Finding the sum and average of an array

```
sum = 0;  
for (index = 0; index < 10; index++)  
    sum = sum + sales[index];
```

```
average = sum / 10;
```

- Largest element in array

```
maxIndex = 0;  
for (index = 1; index < 10; index++)  
    if (sales[maxIndex] < sales[index])  
        maxIndex = index;  
largestSale = sales[maxIndex];
```

Implementing arrays

- Array Index Out of Bounds

- If we have the statements:

```
double num[10];  
int i;
```

- The component `num[i]` is a valid index if `i = 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9`
 - The index of an array is in bounds if the `index >= 0` and the `index <= ARRAY_SIZE-1`
 - If either the `index < 0` or the `index > ARRAY_SIZE-1`
 - then we say that the `index` is out of bounds
 - There is no guard against indices that are out of bounds
 - C++ does not check if the index value is within range

Implementing arrays

- Restrictions on Array Processing

- `int mylist[5]={1,2,3,4,5};`

- `int yourlist[5];`

Assignment does not work with arrays

- `yourlist=mylist;` `||` illegal

In order to copy one array into another array we must copy component-wise

- `for(int i=0;i<5;i++)`

- `yourlist[i]=mylist[i];`

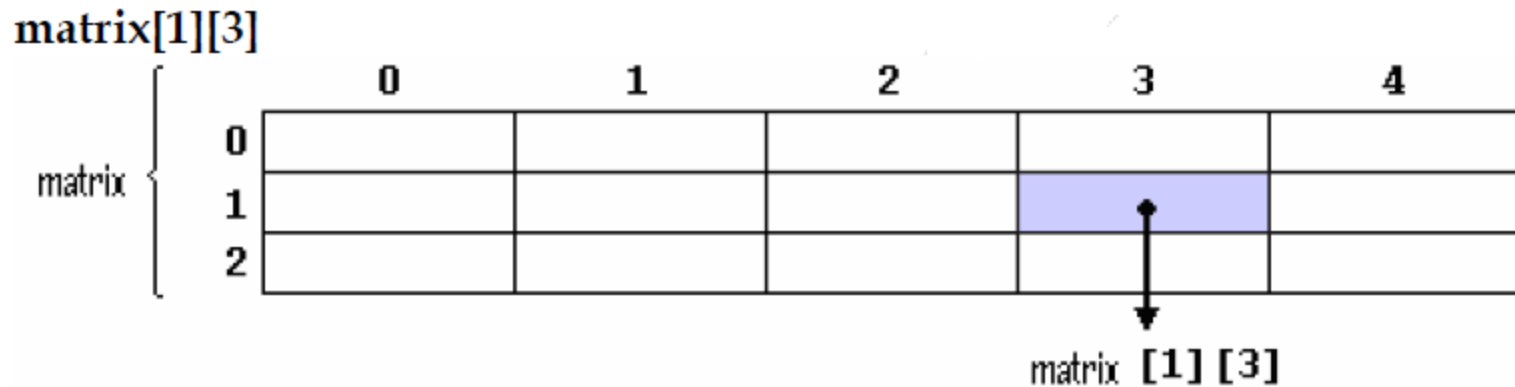
- `cin>>yourlist;` `||` illegal, in stead do the following

- `for(int i=0;i<5;i++)`

- `cin>>yourlist[i];`

Multidimensional arrays

- Multidimensional arrays can be described as arrays of arrays. For example, a bi-dimensional array can be imagined as a bi-dimensional table of a uniform concrete data *type*



- Matrix represents a bi-dimensional array of 3 per 5 values of type **int** . **The way to declare this array would be: `int matrix[3][5];`**
- For example, the way to reference the second element vertically and fourth horizontally in an expression would be: **`matrix[1][3]`**

Multidimensional arrays

- **Initializing multidimensional arrays.**

- `int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };`

- `int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };`

- `int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };`

- Values in array1 by row are:

- 1 2 3

- 4 5 6

- Values in array2 by row are:

- 1 2 3

- 4 5 0

- Values in array3 by row are:

- 1 2 0

- 4 0 0

Multidimensional arrays

- **Initializing multidimensional arrays.**

```
#include <iostream>

void printArray( const int [][][ 3 ] );

int main() {
int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };
cout << "Values in array1 by row
are:" << endl;
```

```
printArray( array1 );
cout << "\nValues in array2 by row
are:" << endl;
printArray( array2 );
cout << "\nValues in array3 by row
are:" << endl;
printArray( array3 );
return 0;
} // end main

void printArray( const int a[][ 3 ])
{ // loop through array's rows
for ( int i = 0; i < 2; i++ ) {
for ( int j = 0; j < 3; j++ )
cout << a[ i ][ j ] << ' ';
cout << endl; // start new line of
output
} // end outer for
}
```

Multidimensional arrays

Exercise

- Write a C++ program that reads through real matrix, 10x10 dimensioned and finds the smallest element in diagonal of the matrix and computes the sum of the diagonal elements.
- Write a C++ program that transposes matrix. Program stores given matrix dimensions and every single matrix element must be given. Transposed matrix is the one with rows and columns switched.
- Write your own C program that stores real matrix whose dimensions are 10x10, and finds the sum of elements from every column and product of elements from every row. Program prints the smallest sum (including parent column's index), and biggest product (including parent row's index). Sums and products should be stored in one-dimensional arrays.

Strings and arrays of characters

- ***What are Strings?***
- In all programs and concepts we have seen so far, we have used only numerical variables, used to express numbers exclusively. But in addition to numerical variables there also exist strings of characters that allow us to represent successive characters, like words, sentences, names, texts, etc. Until now we have only used them as constants, but we have never considered variables able to contain them.
- In C++ there is no specific *elementary variable type to store string of characters*. In order to fulfill this feature we can use arrays of type char, which are successions of char elements. Remember that this data type (char) is the one used to store a single character, for that reason arrays of them are generally used to make strings of single characters.

Strings and arrays of characters

- For example, the following array (or string of characters) can store a string up to 20 characters long. You may imagine it thus: **char name [20];**
- Since the array of characters can store *shorter strings than its total length*, there has been reached a convention to end the valid content of a string with a null character, whose constant can be written as **'\0'**.
- We could represent **name** (an array of 20 elements of type char) storing the strings of characters "Hello" and "Studying C++" in the following way:
 - H e l l o \0

Strings and arrays of characters

String initialization

- `char mystring [] = { 'H', 'e', 'l', 'l', 'o', '\0' };`
- `Char mystring [] = "Hello";`

- **Assigning Values to Strings**

- Just like any other variables, array of character can store values using assignment operators. But the following is not allowed.

- `mystring="Hello";`
- `mystring [] = { 'H', 'e', 'l', 'l', 'o', '\0' };`

- ***Solution for assignment***

- *Use indexes:* `mystring[0]='h';mystring[1]='e',...`
- *Use strcpy:* `strcpy(mystring,"hello");`
- include `<string.h>` header to use `strcpy()` function

Strings and arrays of characters

- cin, cin.getline(), cin.get()

```
#include <iostream>
```

```
void main() {
```

```
    char message[80];                // room for 79 characters
```

```
    cout << "Enter a string:\n";
```

```
    cin.getline(message,80);
```

```
    cout << "The string just entered is :\n";
```

```
    cout << message << '\n';
```

```
}
```

- cin.getline() will read up to the number of characters specified. cin.getline() must be used to read a string. cin will stop reading as soon as it sees the first space or newline.
- The general form of cin.getline() is:
cin.getline(str, length, char)
- where str is a string or character pointer variable, length is an integer constant or variable indicating the maximum number of input characters, and char is an optional character constant or variable specifying the terminating character. If this optional third argument is omitted, the default terminating character is the newline ('\n') character.

Strings and arrays of characters

`cin>>`

With simple data types (int, char, float)

- Reads one symbol at a time
- Skips over white space characters (space, tab, new line, etc ...)

With character arrays (char [])

- Reads one word at a time
- Skips over white space characters (space, tab, new line, etc ...)
- To read in a phrase, need to call `cin>>` several times

`cin.get( )` function

- Reads in a single character at a time
- Reads ALL characters, including white space.

`cin.getline( )` function

- Reads in a string of several words, including the spaces
- Used only with character arrays
- Stops when “terminating character” is reached or array is full

Strings and arrays of characters

Functions to manipulate strings

- The cstring library (`string.h`) defines many functions to perform some manipulation operations with C-like strings (like already explained `strcpy`). Most frequently used string functions

a) String length

- Returns the length of a string, not including the null character (`\0`).
`strlen (const char* string );`

b) String Concatenation:

- Appends *src string at the end of dest string. Returns dest.*
- The string concatenation can have two forms, where the first one is to append the whole content of the source to the destination the other will append only part of the source to the destination.
- Appending the whole content of the source
`strcat (char* dest , const char* src );`
- Appending part of the source
`strncat (char* dest , const char* src, int size );`
 - » Where *size* is the number characters to be appended

c) String Copy:

- Overwrites the content of the *dest* string by the *src* string. Returns *dest*.
- The string copy can have two forms, where the first one is to copying the whole content of the source to the destination and the other will copy only part of the source to the destination.
- Copy the whole content of the source
`strcpy (char* dest , const char* src );`
- Appending part of the source
`strncpy (char* dest , const char* src, int size );`
 - Where *size* is the number characters to be copied

d) String Compare:

- Compares the two string *string1* and *string2*.
- The string compare can have two forms, where the first one is to compare the whole content of the two strings and the other will compare only part of the two strings.

Strings and arrays of characters

Strcmp()

- Compare the whole content of the source
`strcmp (const char* string1 , const char* string2 );`
- compare part of the source
`strncmp (const char* string1 , const char* string2, int size );`
 - Where size is the number characters to be compared
- Both string compare functions returns three different values:
 - Returns **0** is the strings are equal
 - Returns **negative** value if the first is less than the second string
 - Returns **positive** value if the first is greater than the second string

Passing arrays to function

- You can pass array to a function.
- However, it is impossible to pass arrays to a function by value. In other words, it is not possible to pass by value a complete block of memory as a parameter, even if it is ordered as an array, to a function.
- You can pass array to a function by reference. it is allowed to pass its address, which has almost the same practical effect and is a much faster and more efficient operation.
- In order to admit arrays as parameters the only thing that we must do when declaring the function is
 - *to specify in the argument the base type for the array that it contains,*
 - an identifier and
 - a pair of void brackets [] .
 - For example: void procedure (int arg[])

- Here you have a complete example:

```
#include <iostream>
```

```
#include <conio.h>
```

```
void printarray (int arg[], int length) {
```

```
    for (int n=0; n<length; n++)
```

```
        cout << arg[n] << " ";
```

```
        cout << "\n";
```

```
}
```

```
void mult(int arg[], int length) {
```

```
    for (int n=0; n<length; n++)
```

```
        arg[n]=2*arg[n];
```

```
}
```

```
void main () {  
    int firstarray[] = {5, 10, 15};  
    int secondarray[] = {2, 4, 6, 8, 10};  
    printarray (firstarray,3);  
    printarray (secondarray,5);  
    mult(firstarray,3);  
    cout<<"first array after being doubled is\n";  
    printarray (firstarray,3);  
    getch() ;  
}
```